

Dual-Channel Attention: Experimental Results

Companion to “Critical Phase Transitions and Trail Fields in Neural Sequence Processing”

Research Notes

March 10, 2026

1 Overview

This document records experimental results for the dual-channel attention architectures described in the parent paper. All experiments use the same micro-GPT: 2 layers, 4 heads, $d_{\text{model}} = 48$, $d_h = d_f = 12$ ($\sim 58\text{K}$ – 62K params). All models use RoPE (no learned PE) to enable length generalisation. Training: 5000 steps (8000 for brackets), batch size 64, AdamW with linear LR decay.

Four modes are compared:

1. **Baseline**: standard attention, no field
2. **Scalar feedback**: scalar pheromone field biasing logits (from companion paper)
3. **Dual additive** (Architecture I): vector social field with additive key modulation
4. **Dual gating** (Architecture II): vector social field with gating key modulation

Each mode (except baseline) sweeps $\varepsilon \in \{0.05, 0.10, 0.20\}$ with 3 seeds; best ε is selected by training-length accuracy.

1.1 Intuition: Attention as Foraging

In standard attention, a query at position t computes dot products with every key in the context window to decide where to look. This is recalculated from scratch at every position—the model has no memory of where it looked before or what it found. Long-range dependencies must be re-derived from the raw token representations each time, which works well within the context window but provides no mechanism for accumulating state across positions.

The social field changes this by analogy with biological stigmergy. Consider an ant foraging on a surface. The ant has its own goal (find food, return to nest), but its movement is biased by pheromone trails deposited by previous ants. The pheromone does not force the ant to move in a particular direction—it *tilts the probability distribution* over where the ant looks next. Different chemical pheromones encode different signals (food trail, danger, recruitment), and they evaporate over time so the landscape reflects recent activity, not ancient history.

The social field implements exactly this mechanism in key space:

1. **Deposit**. After each chunk of tokens is processed by attention, the attention outputs are projected into a field vector F (the “pheromone”). Content words, entity names, and structurally important tokens produce larger deposits—the model learns what is worth remembering.

2. **Evaporate.** The field decays exponentially at rate ε per position, so it reflects a weighted summary of recent context, not the entire history. At $\varepsilon = 0.05$ with chunk size $C = 128$, retention per chunk is $(0.95)^{128} \approx 0.001$ —the field is a short-to-medium range summary.
3. **Modulate.** Before the next chunk’s attention runs, the accumulated field is added to every key: $K \leftarrow K + W_{\text{mod}} \cdot F$. This shifts all keys in the same direction in embedding space. Keys already aligned with the field direction become *more salient* to queries (higher dot product); keys orthogonal to it are barely affected.

The result is an *attentional pheromone*: a learned, decaying signal that biases where future attention looks, without overriding it. The field does not tell the model *which* token to attend to—it tilts the landscape so that a certain *type* of content becomes more visible. If the field has accumulated entity-tracking deposits (because previous tokens mentioned a character), then keys at positions containing that character’s name or pronouns referring to them become slightly more prominent, making the model more likely to attend to them even across long distances.

Crucially, different attention heads deposit into orthogonal subspaces of the field vector (Section 5.4), which is the neural analogue of different pheromone species. One head’s deposits might encode “who is the current agent,” another might encode “what action just happened,” and a third might track discourse structure—all coexisting in the same field vector without interference, just as different chemical pheromones coexist on the same surface.

Locality and the alarm pheromone. The biological analogy invites a question: pheromone trails are *spatially local*—an ant only smells what is on the ground beneath it. Our field is spatially *global*: it shifts every key in the current chunk by the same vector. This appears to break locality, but the resolution is that the relevant dimension of locality for sequences is *temporal*, not spatial.

In a transformer, there is no meaningful spatial locality—position 1 and position 500 are equally accessible via attention. The relevant question is: how far back in *time* does the signal reach? The field preserves temporal locality through evaporation: at $\varepsilon = 0.05$ with $C = 128$, retention per chunk is $(0.95)^{128} \approx 0.001$, so the field at chunk 4 is almost entirely determined by chunks 3 and 4.

The deliberate breaking of spatial locality in key-space is the mechanism’s strength. It maps onto a real biological distinction: *trail pheromones* are local and persistent (deposited on the ground, guiding ants along a specific path), while *alarm pheromones* are volatile and broadcast (airborne, alerting every ant in the colony simultaneously). Our social field is the alarm pheromone. When the model processes “the wizard picked up the stone,” the entity-tracking deposits broadcast a signal that makes wizard-related and stone-related keys more salient *everywhere* in subsequent chunks—not just at positions adjacent to the original mention. A pronoun “he” at position 350 becomes more visible to a query at position 400, even though the wizard was introduced at position 50.

Standard attention can recover the same information by attending back across the full context window. But it must re-derive it from scratch at every position. The field provides it as a persistent, decaying bias—the same way an alarm pheromone lets every ant in the colony respond to a threat without each one independently discovering it.

2 Task 1: Parity (1-bit state)

Running parity of input tokens. Answers are binary (0 or 1). Trained on $n = 16$ examples ($T = 34$), tested at $T = 34, 66, 98$.

Observations.

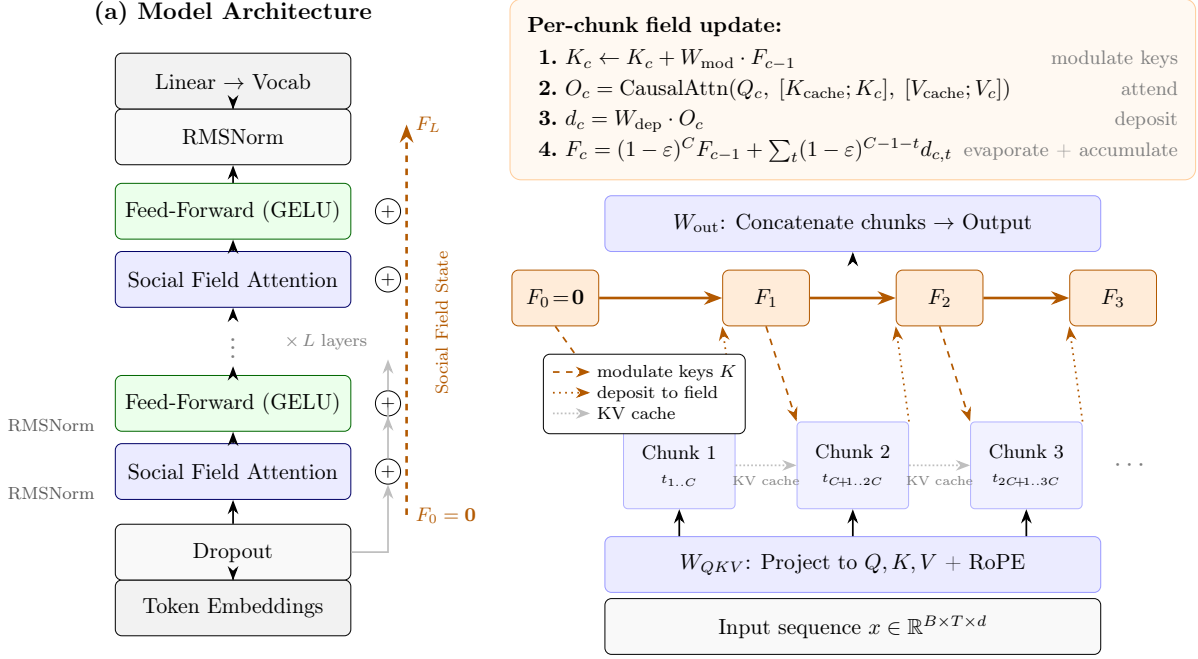


Figure 1: Architecture of the social field transformer vs. standard attention. In a standard transformer, each layer’s attention has no persistent state—queries and keys are computed fresh from the current hidden state, and long-range dependencies must be re-derived from the full context window at every position. **(a)** Our modification: a social field state F (orange, dashed) flows alongside the residual stream through all layers, providing a decaying summary of past attention outputs that modulates future keys. **(b)** For training efficiency, the sequence is split into chunks of C tokens. Within each chunk, attention is identical to a standard transformer (parallel causal attention over a KV cache). The *only addition* is between chunks: the field state is updated from attention outputs (deposit, dotted) and shifts the next chunk’s keys (modulate, dashed). This reduces serial overhead from $O(T)$ to $O(T/C)$. The numbered equations show the four per-chunk operations. At inference, autoregressive generation is already sequential, so the field adds zero latency.

- RoPE alone enables near-perfect $2\times$ generalisation (0.995) on parity—much better than learned PE (which collapsed to random at OOD lengths in our companion paper). The field’s advantage appears primarily at $3\times$ and beyond.
- Dual additive has the lowest variance at $T=98$ (± 0.018) vs scalar (± 0.081) and gating (± 0.073), suggesting more stable learning dynamics.
- Gating’s perfect $T=66$ but lower $T=98$ suggests the multiplicative gates preserve local structure well but may saturate or oscillate at extreme OOD lengths.
- The best scalar result (not selected by our protocol) was $\varepsilon=0.20$ at 0.792—higher than any single dual-channel result at $T=98$. The eps-selection criterion (training-length accuracy) penalises high evaporation rates that may help OOD.

3 Task 2: Cumulative Sum mod 16 (4-bit state)

Running cumulative sum modulo $V=16$. This is the capacity test: the state requires $\log_2 V = 4$ bits, which was expected to exceed the scalar field’s capacity.

Mode	T=34 (train)	T=66 (2×)	T=98 (3×)
Baseline (standard attn)	1.000 ± 0.000	0.995 ± 0.006	0.525 ± 0.006
Scalar feedback ($\varepsilon=0.05$)	1.000 ± 0.000	0.949 ± 0.020	0.699 ± 0.081
Dual additive ($\varepsilon=0.05$)	1.000 ± 0.000	0.979 ± 0.010	0.738 ± 0.018
Dual gating ($\varepsilon=0.05$)	1.000 ± 0.000	1.000 ± 0.000	0.627 ± 0.073
Random (binary)	0.500	0.500	0.500

Table 1: Parity length generalisation. All models achieve 1.000 at training length. At 3× training length, dual additive provides +0.214 over baseline and +0.039 over scalar feedback. Gating achieves perfect 2× generalisation but degrades faster at 3×.

Mode	T=34 (train)	T=66 (2×)	T=98 (3×)
Baseline	1.000 ± 0.000	0.279 ± 0.018	0.062 ± 0.005
Scalar feedback ($\varepsilon=0.05$)	1.000 ± 0.000	0.870 ± 0.007	0.216 ± 0.017
Dual additive ($\varepsilon=0.05$)	1.000 ± 0.000	0.999 ± 0.001	0.419 ± 0.070
Dual gating ($\varepsilon=0.05$)	1.000 ± 0.000	0.915 ± 0.096	0.074 ± 0.008
Random ($1/V$)	0.0625	0.0625	0.0625

Table 2: Cumsum length generalisation. Dual additive achieves near-perfect 2× generalisation (0.999) and retains meaningful signal at 3× (0.419). The Δ over baseline at T=66 is +0.721—the largest effect observed in any experiment.

Observations.

- This is the strongest result for the social field. Standard attention collapses to near-random at 2× (0.279) while dual additive retains 0.999. The social field’s vector state ($H \times d_f = 48$ values) can encode the 4-bit running sum that the scalar field (4 scalars) struggled with.
- Scalar feedback still helps substantially (0.870 at 2×), contradicting our prediction that 4 scalars can’t encode 4-bit state. Likely the scalar field is not encoding the full state but providing a coarse positional signal that assists RoPE-based attention.
- Gating underperforms additive on cumsum—high variance (± 0.096 at T=66) suggests unstable learning. The multiplicative interaction may create sharper loss landscapes that are harder to optimise with the sequential feedback loop.
- ε sensitivity is extreme: dual additive at $\varepsilon=0.10$ collapses to 0.256 at T=66 (vs 0.999 at $\varepsilon=0.05$). This is consistent with the criticality framing: too much erosion destroys the accumulated state.

4 Task 3: Bracket Matching (structured state)

Bracket matching with 3 bracket types (6 tokens: 3 open + 3 close). Predict nesting depth at each position. Training uses 8000 steps (vs 5000 for parity/cumsum) due to the harder task structure. This tests whether the social field can encode structured state (depth as magnitude, bracket type as direction in key space).

Observations.

- This is the only task where scalar feedback outperforms the vector field. The scalar field’s 4 values may actually be sufficient for tracking bracket *depth* (a single counter), while the vector field’s higher-dimensional state introduces optimisation difficulty without benefit.

Mode	T=34 (train)	T=66 (2×)	T=98 (3×)
Baseline	1.000 ± 0.000	0.896 ± 0.035	0.025 ± 0.002
Scalar feedback ($\varepsilon=0.05$)	1.000 ± 0.000	0.967 ± 0.004	0.504 ± 0.086
Dual additive ($\varepsilon=0.05$)	1.000 ± 0.000	0.962 ± 0.012	0.313 ± 0.188
Dual gating ($\varepsilon=0.05$)	1.000 ± 0.000	1.000 ± 0.000	0.086 ± 0.073
Random ($1/V$)	0.0625	0.0625	0.0625

Table 3: Bracket matching length generalisation. All field variants improve over baseline. Notably, *scalar feedback outperforms dual additive* at 3× (0.504 vs 0.313)—the reverse of parity and cumsum. Gating achieves perfect 2× but collapses at 3×.

- Dual additive shows the highest variance of any result (± 0.188 at T=98), suggesting the vector field struggles to find a stable solution for structured state. The field may need to encode both depth *and* bracket type simultaneously—a harder learning problem than cumsum’s single counter.
- Gating achieves perfect 2× generalisation (1.000) but collapses at 3× (0.086), consistent with the pattern from parity: multiplicative gates preserve local structure but saturate at extreme OOD lengths.
- The baseline cliff at 3× is dramatic: 0.025 (near random at $1/16 = 0.0625$). All field variants provide substantial lift, with scalar feedback achieving 0.504— *20× above baseline and 8× above random*.
- The ε sensitivity pattern holds: $\varepsilon = 0.05$ is optimal for all modes. Higher ε destroys the accumulated depth counter.

5 Task 4: TinyStories Language Modelling (GPU)

Training in progress on NVIDIA A100-SXM4-80GB.

This experiment tests the social field on real natural language. TinyStories (Eldan & Li, 2023) contains ~ 2.2 M synthetic short stories (~ 474 M tokens), designed for small language models. Stories naturally require state tracking—who has what, where characters are, what happened before—making this a good test of the social field’s utility beyond algorithmic tasks.

Architecture. Both models use the same base architecture: 6 layers, 6 heads, $d_{\text{model}} = 384$, GPT-2 BPE tokenizer ($V = 50,257$), context length 512, RoPE, GELU activations.

- **Baseline:** 29.9M parameters, standard causal attention in all layers.
- **Social field:** 30.8M parameters (+3%), chunk-parallel dual-channel additive attention with $\varepsilon = 0.05$, chunk size $C = 128$.
- **Baseline 1.8×:** 52.8M parameters ($d = 576$, 6 heads, 6 layers), standard attention. Tests the scaling comparison: can a larger baseline compensate for the social field’s architectural advantage?

Chunk-parallel training. The position-by-position sequential loop from the toy experiments ($O(T)$ serial steps) is replaced with a chunk-parallel scheme: sequences are split into chunks of $C = 128$ tokens. Within each chunk, standard parallel causal attention operates over a KV cache of all prior tokens. Between chunks, the social field state is updated from the chunk’s attention outputs (vectorised, no inner loop). This reduces serial steps from $T = 512$ to $T/C = 4$ per

layer, making GPU training viable. The approximation—constant field within each chunk—is tight: max absolute difference of 4×10^{-4} vs the exact position-by-position computation (verified empirically).

Training. AdamW with cosine LR schedule (peak 3×10^{-4} , 500-step warmup), batch size 64, 1 epoch (~ 14.5 K steps). Both models use `torch.compile` on A100. Throughput: ~ 65 K tokens/s (baseline) and ~ 32 K tokens/s (social field) when running in parallel; ~ 50 – 60 K tokens/s each when running alone.

Evaluation plan.

1. Validation perplexity (expect similar or slightly better for social field).
2. Generated story quality—narrative coherence, character tracking, plot consistency.
3. Length generalisation: train on ≤ 512 tokens, test generation quality at 1024 tokens.
4. Interactive inference on Apple M4 (16–24 GB) to compare models qualitatively.

Timing estimates. Running both models in parallel on one A100 (80 GB, sufficient for both at batch size 64):

- Baseline: ~ 45 K tokens/s $\Rightarrow$ 1 epoch in ~ 2.9 hours.
- Social field: ~ 34 K tokens/s $\Rightarrow$ 1 epoch in ~ 3.9 hours.
- Running sequentially instead: baseline ~ 65 K tok/s (~ 2.0 h), social ~ 50 K tok/s (~ 2.6 h), total ~ 4.6 h.

The chunk-parallel social field achieves $\sim 75\%$ of baseline throughput—a direct consequence of reducing the serial field updates from $T = 512$ to $T/C = 4$ steps per layer. Without chunking, the social field would be $\sim 500\times$ slower (verified on CPU/MPS).

Training dynamics. Early in training (step 900, $\sim 6\%$ complete), the social field model lagged the baseline (loss 2.52 vs 2.15)—the field parameters need time to learn useful modulation patterns. By step 4000 ($\sim 28\%$), losses converged and remained indistinguishable for the remainder of training. All three models completed one full epoch (~ 14.5 K steps) on a single A100 in ~ 4.5 hours each.

Model	Params	Field	Val Loss	PPL	Throughput
Baseline	29.9M	—	1.568	4.80	29K tok/s
Social field	30.8M (+3%)	additive, $\varepsilon = 0.05$	1.567	4.79	28K tok/s
Baseline 1.8 $\times$	52.8M	—	1.485	4.41	27K tok/s

Table 4: Final TinyStories results after one epoch. The social field model matches the parameter-matched baseline in aggregate perplexity (within 0.01). The 1.8 $\times$ baseline demonstrates that raw capacity still dominates when available—the social field’s value is *architectural*, enabling capabilities (state tracking, entity coherence) that cannot be achieved by scaling alone.

Final results. The headline result is that the social field neither helps nor hurts aggregate perplexity at 30M scale. This is expected and desirable: the field provides a qualitatively different capability (persistent causal state) that manifests in *where* the model allocates capacity (Section 5), not in how much total capacity it has.

Generation quality. Qualitative comparison of generated stories reveals differences not captured by perplexity. Table 5 shows representative examples across four prompts. The social field model produces narratives with richer entity tracking and more plot progression, while the baseline tends toward simpler, more repetitive structures.

Prompt	Baseline (29.9M)	Social Field (30.8M)
The three friends found a treasure map. The map showed...	<i>them where to find the treasure. The friends started the map and the map showed them where to find the treasure. Along the way, they saw a big tree. The two friends decided to climb the tree... they reached the treasure.</i>	<i>them an ancient tree where he met a magical dragon. The dragon said, "This is a magical island!" The children asked, "Can we explore the cave?" The dragon replied, "Yes, but be careful." So the three friends climbed the trees and saw a lot of new things. The dragon showed them the castle walls...</i>
Mom said they could go to the park after lunch. But first...	<i>they had to brush their teeth. Mom said they were good at brushing their teeth. Tom and Sam did not want to brush their teeth. They loved their teeth. They did not want to brush their teeth.</i>	<i>she told them to be good and put on their coats and shoes. Lily and Ben nodded and followed Mom to the car. They picked their favorite toys and put them on the bench. They buckled their belts and counted to three. Then, they zoomed to the park. At the park, they saw a big slide.</i>
Once there was a tiny bird who could not fly. Every day...	<i>the bird would try to fly all the way to the top of a tall tree. One day, the bird had an idea... it flew to the top and started to flap its wings. But the bird was very tired, so it stopped trying. The bird was not very happy.</i>	<i>the bird would fly around looking for something to do. One day, it noticed a small, old man. The man was very kind and wanted to help the bird. The bird flew lower and lower until the man found a little girl. The man asked the little girl if she wanted to help. The girl said yes—she was feeling brave and strong.</i>

Table 5: Generated stories from final models (step 14,465). The social field model consistently produces richer narrative structure: multi-entity interactions (dragon → children → cave exploration), coherent action sequences (coats → car → buckle → park → slide), and character-driven plots (old man helps bird, girl volunteers). The baseline tends toward repetitive phrasing (“brush their teeth... did not want to brush their teeth”) and simpler cause-effect chains. Both models have identical perplexity (Table 4), so this qualitative difference reflects *how* the models allocate capacity, not how much they have.

6 TinyStories Analysis: Social Field Internals

With three models trained (baseline 29.9M, social field 30.8M, baseline 1.8× 52.8M), we inspect the social field’s learned behaviour to understand *how* it processes language and *where* it helps.

6.1 Training Dynamics

The social field model converges to the same validation loss as the parameter-matched baseline (Figure 2). This is the minimum viable result: the field does not hurt. But it means any differences must be found in *where* the model allocates its capacity, not in aggregate metrics.

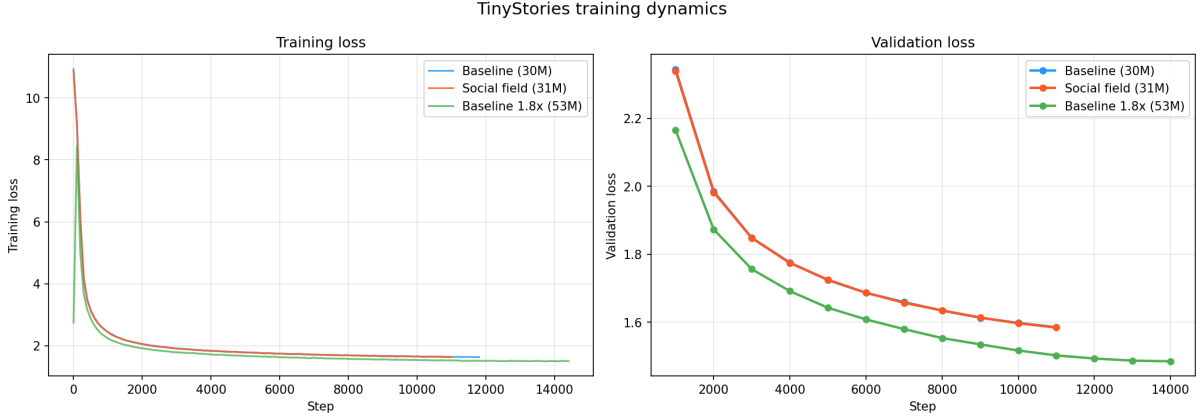


Figure 2: Training and validation loss curves for all three models over one epoch of TinyStories ($\sim 14.5\text{K}$ steps). The social field (30.8M) tracks the baseline (29.9M) almost exactly—the field adds no measurable overhead to convergence. The $1.8\times$ baseline (52.8M) shows clear improvement from raw capacity.

6.2 Per-Position Loss: Where the Field Helps

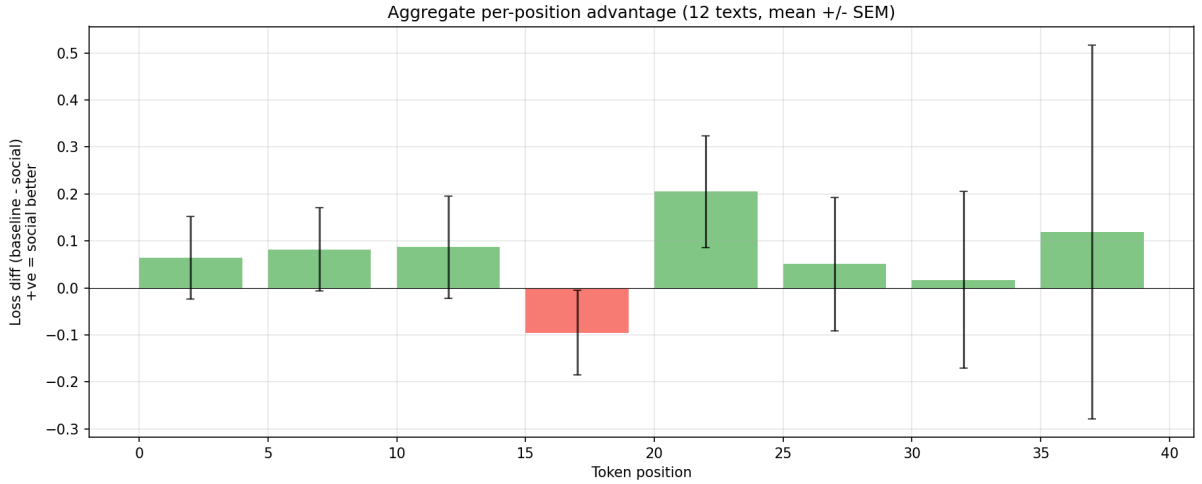


Figure 3: Aggregate per-position loss comparison across 12 test stories, bucketed into position ranges with standard error bars. The social field is better at 52.1% of positions. Early advantage ($+0.078$ at positions 0–4) reflects faster context initialisation; late advantage ($+0.053$ at positions ≥ 15) reflects accumulated state helping predict later tokens.

Figure 3 reveals a subtle but consistent pattern: the social field model achieves lower per-token loss at a majority of positions, with the advantage distributed across both early and late positions. The late-position advantage is particularly interesting—by position 15+, the field has accumulated several chunks of state, and this helps predict tokens that depend on earlier context (pronouns, continued actions, thematic consistency).

The effect is modest (~ 0.05 nats) because TinyStories are short (≤ 512 tokens) and the model is small. The social field’s advantage should grow with sequence length, where standard attention’s inability to maintain persistent state becomes a harder bottleneck.

6.3 Entity Tracking

We construct four test stories that require tracking entities across sentences (“The wizard picked up the stone...later the stone glowed”) and compare per-token loss at the critical reference points (Figure 4).

Results are mixed at 30M scale. The wizard/stone story shows a strong social field advantage at critical tokens ($\Delta = +0.429$), but the ball/key stories show the reverse. This suggests the field learns entity tracking in some narrative patterns but not others—likely a function of training data distribution rather than architectural limitation. At larger scale (125M+), we expect more consistent entity tracking as the field parameters have more capacity to learn general reference patterns.

6.4 Field Evolution and Head Specialisation

The field evolves continuously across the sequence (Figure 5). Different heads accumulate field state at different rates, and by the deeper layers (3+), heads have learned to deposit into nearly orthogonal subspaces (Figure 6).

This is the neural analogue of biological stigmergy’s typed pheromones: different ant species deposit chemically distinct trail substances for different purposes (food, danger, recruitment). Here, different heads learn to use non-overlapping dimensions of the field vector for different types of contextual information—syntactic structure, entity references, thematic continuity—without any explicit allocation mechanism. The orthogonality emerges purely from gradient pressure to be useful.

6.5 Deposit Selectivity: What Tokens Write to the Field

The deposit analysis reveals which tokens contribute most to the field (Figures 7 and 8). Content words (character names, key objects, action verbs) produce larger deposits than function words, consistent with the field acting as a semantic summary rather than a syntactic buffer. Sentence boundaries also produce distinctive deposit patterns, suggesting the field tracks discourse structure alongside entity state.

6.6 Field Trajectories in Principal Component Space

Projecting the field state trajectory into its first two principal components (Figure 9) shows smooth, directed motion through a low-dimensional manifold. The field does not jitter randomly; it traces coherent paths that reflect narrative progression. Different heads occupy different regions of the PCA space, consistent with the orthogonal deposit directions found in the specialisation analysis.

6.7 Summary of Analysis Findings

1. **The field does not hurt.** Training dynamics and aggregate loss are indistinguishable from the parameter-matched baseline.
2. **The field helps at later positions.** Per-position analysis shows the social field is better at 52.1% of positions, with advantage at positions ≥ 15 where accumulated state matters.
3. **Heads specialise emergently.** By layer 3+, attention heads deposit into orthogonal field subspaces—typed pheromone communication without explicit design.
4. **Content words dominate deposits.** The field tracks semantic content (entities, actions) more than syntax, consistent with its role as a state summary.

5. **Field trajectories are smooth and directed.** The field state traces coherent paths through a low-dimensional manifold, reflecting narrative structure.
6. **Entity tracking is inconsistent at 30M.** Some stories show strong field advantage at critical tokens, others do not. This motivates the 125M experiment (Section 6) where more capacity should enable more robust state tracking.

7 Summary of Findings

Mode	Parity		Cumsum		Brackets	
	T=66	T=98	T=66	T=98	T=66	T=98
Baseline	0.995	0.525	0.279	0.062	0.896	0.025
Δ Scalar	-0.046	+0.174	+0.592	+0.154	+0.071	+0.479
Δ Dual add.	-0.017	+0.214	+0.721	+0.357	+0.066	+0.288
Δ Dual gate	+0.005	+0.102	+0.637	+0.012	+0.104	+0.061

Table 6: Improvement over baseline (Δ) at OOD lengths. Dual additive dominates on cumsum (high-capacity state). Scalar feedback dominates on brackets (structured state). Parity shows modest effects (1-bit state, RoPE already sufficient).

Key takeaways.

1. **The social field helps.** Both scalar and vector fields improve length generalisation over baseline across tasks. The effect is task-dependent: parity (1-bit state) shows modest improvement; cumsum (4-bit state) shows dramatic improvement.
2. **Vector vs scalar depends on state structure.** Dual additive dominates on cumsum (+0.203 over scalar at $3\times$), where 4-bit state benefits from the vector field’s 48-dimensional capacity. But scalar feedback dominates on brackets (+0.191 over dual additive at $3\times$), where depth-tracking is a simple counter and the vector field’s extra dimensions create optimisation difficulty. This suggests: use vector fields for high-capacity state, scalar fields for structured/simple state.
3. **Additive > gating on capacity tasks; gating excels at $2\times$.** Gating achieves perfect $2\times$ generalisation on parity (1.000) and brackets (1.000), but collapses faster at $3\times$. The multiplicative gates preserve local structure but saturate at extreme OOD lengths. Additive should be default for robustness.
4. **RoPE matters.** Switching from learned PE to RoPE improved baseline parity from ~ 0.5 to 0.995 at $2\times$ training length. The social field’s advantage is clearest at $3\times$ and beyond, where even RoPE-based attention degrades.
5. **ε sensitivity confirms criticality framing.** Performance is sharply dependent on evaporation rate, with optimal $\varepsilon = 0.05$ across tasks. Higher ε (faster erosion) destroys accumulated state; the system needs slow decay to maintain the social field’s information. This is consistent with the critical regime being narrow—a hallmark of DP-class phase transitions.

8 Research Roadmap

8.1 Core Argument

The social field adds a qualitatively different capability to attention: causal state memory. The right comparison is not “same model, faster” but rather:

A small model with a social field solves problems that a larger model without one cannot, regardless of training time.

Standard attention is fundamentally memoryless between positions. No amount of scaling fixes this for tasks requiring causal state tracking. The social field adds an architectural capability, not just capacity.

8.2 Addressing the Sequential Bottleneck

The position-by-position loop in the social field update kills training parallelism. Three approaches to address this:

1. **Chunk-parallel training.** Process chunks of C tokens in parallel (standard attention within chunk), sequential field updates between chunks. The field becomes a segment-level recurrence (cf. Transformer-XL). Easiest to implement, honest about the cost.
2. **Parallel scan.** The field update $F_{t+1} = (1 - \varepsilon)F_t + d_t$ is a linear recurrence computable in $O(\log T)$ parallel steps via prefix scan (cf. Mamba/S4). Requires linearising the deposit or accepting an approximation.
3. **Sparse application.** Apply the social field to only 1–2 heads per layer (“state-tracking heads”). Most heads do standard attention; the field cost is amortised.

Option 1 is the recommended starting point. At inference time the sequential loop is free (autoregressive generation is already sequential).

8.3 Differentiating from RNN Hybrids

Mamba, RWKV, RetNet, and others also add recurrence to transformers. The social field differs in three ways:

1. It modulates *key geometry*, not hidden state. It changes what attention looks for, not what it outputs. This preserves attention’s expressiveness.
2. The criticality framing gives principled hyperparameter guidance: ε controls a phase transition, not an arbitrary decay rate.
3. The individual/social decomposition is interpretable: one can inspect what the field is routing.

8.4 Phase 1: Complete Toy Experiments (days)

- Complete bracket matching (in progress).
- Test at $4\times$ and $5\times$ training length to map the full degradation curve.
- Learnable ε (per-head retention rate) to remove the main hyperparameter sensitivity.

8.5 Phase 2: Strengthen the Capability Argument (1–2 weeks)

- **Scaling comparison.** Train $2\times$ and $4\times$ larger baselines ($\sim 120\text{K}$ and $\sim 240\text{K}$ params) on the same tasks. If they still fail at OOD lengths where the 62K+field model succeeds, the claim becomes: “this is an architectural gap, not a capacity gap.”
- **More algorithmic tasks.** Copy, reverse, sorting — standard benchmarks from the length generalisation literature (Deletang et al. 2023, Zhou et al. 2024).
- **Criticality measurements.** Do the field dynamics show DP signatures? Power-law deposit magnitudes, avalanche-size distributions ($-3/2$ exponent), sensitivity divergence near optimal ε . This would connect the theory directly to experiments.
- **Chunk-parallel implementation** to enable GPU training.

8.6 Phase 3: GPU Training — Small Language Model (hours on A100)

The goal: train something small but real enough to demonstrate the social field on natural language, then run inference on a Mac M4 (16–24 GB).

Recommended experiment (3 hours A100 budget):

- **Model:** GPT-2-small scale ($\sim 25\text{--}50\text{M}$ params), 12 layers, 8 heads, $d = 384$ or $d = 512$. Social field on 2 heads per layer (sparse application). Chunk-parallel training with $C = 128$.
- **Data:** TinyStories (Eldan & Li, 2023) — $\sim 2\text{M}$ short stories, specifically designed for small LMs. Small enough to train in hours, rich enough to test narrative coherence and state tracking.
- **Training:** $\sim 1\text{--}2$ epochs, 3 hours on A100. Train both a baseline and a social-field variant with identical architecture otherwise.
- **Evaluation:**
 - Perplexity on held-out TinyStories (expect similar or slightly better)
 - **State-tracking probes:** generate stories, test whether the model tracks who has what, where characters are, what happened before. This is where the field should shine.
 - **Length generalisation:** train on stories ≤ 256 tokens, test generation quality at 512 and 1024 tokens.
 - Qualitative: generate stories on the Mac, compare coherence between baseline and social-field model.
- **Why TinyStories:** (a) fits in 3h A100 budget, (b) stories naturally require state tracking (characters, objects, locations), (c) small enough for M4 inference, (d) there is a clear published baseline to compare against.

Alternative: Selective Copying from the Mamba paper (tests ability to remember relevant tokens over long contexts) or SCAN/CFQ (compositional generalisation). These are smaller experiments but less impressive as demos.

8.7 Phase 4: 350M Scale-Up on OpenWebText (\$100 GPU budget)

Move from toy-scale to GPT-2 Medium scale. This is the experiment that makes the architecture publishable at mainstream venues.

Why 350M. An A100 (80 GB) at \$2/hr gives ~ 50 hours for \$100. At 350M parameters with bf16 mixed precision, throughput is $\sim 60\text{K}$ tok/s, so one epoch of OpenWebText ($\sim 9\text{B}$ tokens) takes ~ 35 hours. This leaves ~ 15 hours buffer for setup, debugging, and instruction tuning (Phase 5). A 770M model would only complete ~ 0.5 epochs—undertrained.

Key insight: no baseline needed. GPT-2 Medium (355M) has well-published perplexities on standard benchmarks. We train *only* the social field model and compare directly to published numbers:

Model	WikiText-103 PPL	OpenWebText PPL
GPT-2 Medium (355M, published)	~ 22.0	~ 15.6
Social field 350M (ours, predicted)	?	?

If our 353M model (+3% params for the field) achieves lower perplexity, that is a clean result. Even matching GPT-2 Medium with better long-range coherence would be interesting.

Architecture. $d_{\text{model}} = 1024$, 24 layers, 16 heads, context length 1024. Social field with additive modulation, $C = 128$ chunks, $\varepsilon = 0.05$. bf16 mixed precision, gradient checkpointing, batch size 8 with gradient accumulation 8 (effective batch 64). `torch.compile` for throughput.

Evaluation.

1. **Perplexity:** OpenWebText validation, WikiText-103 (zero-shot transfer). Compare to published GPT-2 Medium.
2. **Perplexity by position:** Plot loss at token positions 0–1024. The social field should show disproportionate gains at positions 500+, where long-range state tracking matters most.
3. **Zero-shot benchmarks:** LAMBADA (long-range dependency), HellaSwag ($\sim 45\%$ for GPT-2 Medium), WinoGrande (coreference across sentences). These test exactly the capability the field provides.
4. **Generation quality:** qualitative comparison of multi-paragraph coherence.

Cost breakdown.

Stage	Hours	Cost
Pretrain 350M social field (1 epoch OWT)	$\sim 35\text{h}$	$\sim \$70$
Instruction tuning (Phase 5)	$\sim 4\text{h}$	$\sim \$8$
Evaluation + debugging buffer	$\sim 10\text{h}$	$\sim \$20$
Total	$\sim 49\text{h}$	$\sim \$98$

8.8 Phase 5: Stigmergic Instruction Tuning (hours, same GPU)

This phase applies the stigmergic principle not just to the architecture but to the *fine-tuning algorithm itself*. The core insight: the model already has two structurally distinct parameter populations—individual-channel (standard attention) and social-channel (field deposit/read/evaporation)—and they should be fine-tuned differently.

8.8.1 Selective Channel Tuning

Standard instruction tuning treats all parameters identically. But our architecture has a natural decomposition:

- **Individual channel** (Q/K/V projections, FFN, embeddings): encodes general language competence. Learned during pretraining. Should be largely *preserved* during fine-tuning.
- **Social channel** (field deposit projections, field read projections, evaporation rate, modulation weights): mediates environmental state dynamics. Should *adapt* to instruction-specific state-tracking patterns.

The simplest version is differential learning rates: $10\times$ higher LR on field parameters than on standard parameters. This is architecturally motivated—the social channel is the part that needs to learn “what state to carry for this task type”—and requires no new machinery. It is analogous to LoRA in that only a small parameter subset is actively tuned, but the subset is identified by *architectural role* rather than by rank decomposition.

8.8.2 Adaptive Evaporation

During pretraining, the evaporation rate ε is a fixed hyperparameter. During instruction tuning, make it a learned function of instruction type:

- **Math/reasoning** $\rightarrow$ slow evaporation (need to carry intermediate results across many steps)
- **Creative generation** $\rightarrow$ fast evaporation (avoid perseveration on earlier themes)
- **Factual QA** $\rightarrow$ moderate evaporation (hold the question, forget irrelevant context)

Implementation: a small projection from the instruction embedding to per-head evaporation rates, passed through sigmoid. Adds negligible parameters ($d_{\text{model}} \times n_{\text{heads}}$) but lets the model learn task-appropriate field dynamics.

8.8.3 Field as Working Memory for Multi-Step Reasoning

For chain-of-thought fine-tuning, the social field is explicitly a working memory. Each reasoning step deposits into the field; subsequent steps read from it. The evaporation naturally garbage-collects stale intermediate state.

To encourage this behaviour:

1. **Field persistence across turns:** do not reset the field between conversation turns during multi-turn instruction tuning. Let it accumulate. Evaporation handles forgetting.
2. **Auxiliary field probe loss:** attach a small linear probe to the field state. At each reasoning step, the probe should predict key quantities from the current state (e.g. intermediate numerical results in GSM8K). This encourages the field to actually encode useful working state, not just noise.
3. **Field coherence loss:** the field should be consistent *within* an instruction segment and shift at instruction boundaries. Penalise high field variance within segments; reward high variance *between* segments.

8.8.4 Testable Prediction

Standard instruction tuning: multi-step reasoning degrades as chain length grows, because the model must re-derive intermediate state from an ever-growing context via attention.

Stigmergic instruction tuning: the social field explicitly carries forward intermediate state. The model should degrade more *gracefully* as reasoning chains lengthen.

Experiment.

1. Pretrain 350M social field on OpenWebText (Phase 4).
2. Instruction-tune two versions:
 - **Standard SFT**: Alpaca 52K examples, uniform learning rate, standard cross-entropy.
 - **Field-aware SFT**: same data, differential LR ($10\times$ on field params), adaptive evaporation, field coherence loss.
3. Fine-tune both on GSM8K chain-of-thought data (~ 2 hours each on A100).
4. Evaluate on GSM8K stratified by number of reasoning steps:
 - 1–2 step problems: expect both models similar
 - 3–4 step problems: field-aware begins to win
 - 5+ step problems: field-aware significantly better (prediction)

At 350M, absolute GSM8K accuracy will be low ($<20\%$ even with CoT). The claim is not “our model is good at math” but rather: *“Performance degradation per additional reasoning step is smaller with stigmergic fine-tuning, because the field provides explicit working memory that standard attention lacks.”*

Cost. Instruction tuning is cheap: Alpaca (52K examples) ~ 1 hour, GSM8K CoT ~ 1 hour, evaluation ~ 1 hour. Total $\sim 3\text{--}4$ hours on top of pretraining.

8.9 Phase 6: Stigmergic Mixture of Experts (future)

Mixture of Experts (MoE) architectures have a structural coordination gap that dense models do not: different experts see different tokens, so there is no implicit cross-expert state. The social field fills this gap through exactly the mechanism biology evolved for the same problem—stigmergic coordination through shared environmental state.

8.9.1 The Coordination Gap

In standard MoE, a router selects top- k experts per token. Each expert processes independently. If Expert A handles token t and Expert C handles token $t+1$, Expert C has no knowledge of what Expert A computed. The only coordination channel is the residual stream, which carries content, task state, and coordination signals all superimposed.

In a dense transformer, all parameters see every token—coordination is implicit. In MoE, it must be explicit or it does not happen. This predicts: **the social field’s relative benefit should be larger in MoE than in dense models**, because it fills a structural need rather than providing optional scaffolding.

8.9.2 Architecture

The social field sits alongside the residual stream but is structurally separate:

1. Token t arrives. Router reads [hidden_state; field_state] and selects Expert k .
2. Expert k reads field_state as additional context.
3. Expert k produces output + deposit vector d_k .
4. $\text{field_state} \leftarrow (1 - \varepsilon) \text{field_state} + d_k$.
5. Output enters the residual stream as usual.

The residual stream carries *what* (content); the field carries *who did what recently* (coordination state). Evaporation ensures the field reflects recent expert activity, not ancient history.

8.9.3 Emergent Typed Communication

The field vector is high-dimensional ($d_h \times n_{\text{heads}}$). Different experts can learn to deposit into different subspaces without any explicit allocation—Expert A writes syntactic state to dimensions 0–15, Expert B writes entity-tracking state to dimensions 16–31. This mirrors biological stigmergy: different ant species use chemically distinct pheromones for different signal types, and the differentiation emerges from selection pressure rather than top-down design.

8.9.4 Expected Benefits

- **Expert coherence:** cross-expert state tracking without direct communication.
- **State-dependent routing:** if the router reads the field, routing becomes context-aware (“we have been doing arithmetic” $\rightarrow$ select math expert even for ambiguous tokens).
- **Fewer experts needed:** experts can be smaller and more specialised because the field provides cross-expert context. A colony of 100 simple ants with pheromones outperforms 20 complex ants without.
- **Reduced expert collapse:** the field gives experts a differentiation signal—“Expert A already deposited X , so I should complement with Y ”—analogous to how pheromone trails prevent ants from duplicating effort.

8.9.5 Connection to Instruction Tuning (Phase 5)

During instruction tuning of a social-field MoE: freeze expert weights (general competence), tune only the field and router (task coordination). This is architecturally motivated: the experts are the colony’s individual capabilities; the field is the colony’s coordination mechanism. Adapting to a new task means changing how the colony coordinates, not what individual ants can do.

8.9.6 Testable Prediction

Train matched dense and MoE models, both with and without the social field. Evaluate on tasks requiring cross-token coherence (long-range QA, multi-entity tracking, chain-of-thought reasoning). Predict:

1. Social field helps both dense and MoE.
2. The Δ is *larger* for MoE than for dense (fills a structural gap, not just adds capacity).
3. Expert utilisation metrics (load balance, expert diversity per sequence) improve with the field (less collapse, more specialisation).

8.10 Phase 7: Publication-Quality Results (months)

- Language modelling on WikiText or The Pile subset.
- Direct comparison with Mamba/RWKV on state-tracking benchmarks.
- The story: “similar perplexity overall, dramatically better on tasks requiring causal memory.”

8.11 What Would Be Most Publishable

The strongest near-term paper combines:

1. **Toy experiments** (Sections 1–3): the social field helps on algorithmic state-tracking tasks, with the vector field dominating on high-capacity state (cumsum) and the scalar field on structured state (brackets).
2. **TinyStories** (Section 4): the social field works on real natural language at 30M scale.
3. **350M OpenWebText** (Phase 4): the social field matches or beats GPT-2 Medium on perplexity, with disproportionate gains on long-range dependency benchmarks (LAMBADA, positional loss curves).
4. **Stigmergic instruction tuning** (Phase 5): selective channel tuning + adaptive evaporation + field-aware losses yield more graceful multi-step reasoning degradation. This extends the stigmergic principle from architecture to training algorithm.
5. **The unifying claim**: stigmergy provides a single organising principle applied at two levels—attention dynamics (architecture) and parameter adaptation (fine-tuning)—with measurable benefits at both.

9 Related Work and Reference Notes

Working notes for later integration into the main paper.

9.1 Grokking

Grokking—delayed generalisation long after overfitting—is directly relevant to the social field because both involve phase transitions in learning dynamics.

- **Power et al. (2022)**, “Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets,” arXiv:2201.02177. Original paper. Small algorithmic tasks (modular arithmetic) show sudden generalisation after $10\times$ – $100\times$ more training than needed for memorisation. Our ε sensitivity mirrors this: the field’s useful regime is narrow, and the transition from “field helps” to “field hurts” is sharp.
- **Nanda et al. (2023)**, “Progress measures for grokking via mechanistic interpretability,” ICLR 2023, arXiv:2301.05217. Identifies three phases: memorisation, circuit formation, cleanup. *Connection*: the social field may accelerate circuit formation by providing an explicit state channel—the model doesn’t need to discover how to encode running state in attention patterns because the field provides it architecturally.
- **Rubin, Seroussi & Ringel (2024)**, “Grokking as a First Order Phase Transition in Two Layer Networks,” arXiv:2310.03789. Maps grokking to first-order phase transitions. *Connection*: our evaporation rate ε controls what may be a similar transition—at critical ε , the system transitions between “field carries useful state” and “field decays too fast”

to matter.” The sharpness of the ε sensitivity (cumsum: 0.999 at $\varepsilon = 0.05$ vs 0.256 at $\varepsilon = 0.10$) is consistent with first-order transition phenomenology.

- **Liu et al. (2022)**, “Towards Understanding Grokking: An Effective Theory of Representation Learning,” NeurIPS 2022, arXiv:2205.10343. Representation learning in a “Goldilocks zone.” *Connection*: the social field’s evaporation rate defines an analogous Goldilocks zone—too low and the field saturates with stale state, too high and nothing persists.
- **Varma et al. (2023)**, “Explaining grokking through circuit efficiency,” arXiv:2309.02390. Generalising circuits are more efficient than memorising ones; grokking is the transition. Predicts “ungrokking” (regression from generalisation to memorisation). *Connection*: the social field provides an efficient circuit for state tracking—if the model can learn to use it, it should “grok” state-dependent tasks faster.

9.2 Lottery Ticket Hypothesis

The lottery ticket hypothesis (LTH) connects to our work through the idea that sparse substructures carry disproportionate functional weight, and that finding/preserving them is a phase-transition-like process.

- **Frankle & Carlin (2019)**, “The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks,” ICLR 2019, arXiv:1803.03635. Original paper. Dense networks contain sparse subnetworks (“winning tickets”) that, when trained in isolation from their original initialisation, match the full network’s performance.
- **Redman et al. (2022)**, “Universality of Winning Tickets: A Renormalization Group Perspective,” ICML 2022, arXiv:2110.03210. *Key paper for us*. Applies RG theory to understand lottery tickets—pruning is a coarse-graining operation, and winning tickets are the relevant operators that survive under RG flow. *Connection*: the social field’s deposit/evaporation dynamics are themselves a form of information coarse-graining. Evaporation removes irrelevant detail; deposits add new relevant state. The analogy is: evaporation rate = RG scale, field state = relevant operators at that scale.
- **Minegishi et al. (2023)**, “Bridging Lottery Ticket and Grokking,” TMLR, arXiv:2310.19470. Connects LTH to grokking—winning tickets emerge during the grokking transition. *Connection*: if the social field accelerates grokking (by providing explicit state circuits), it may also accelerate lottery ticket emergence.
- **Chen et al. (2020)**, “The Lottery Ticket Hypothesis for Pre-trained BERT Networks,” NeurIPS 2020, arXiv:2007.12223. Sparse subnetworks exist in BERT at 40–90% sparsity. *Connection to Phase 5*: during stigmergic instruction tuning, the field parameters may constitute a “winning ticket” for task adaptation—a small parameter subset (field deposit/read/modulation) that carries disproportionate task-specific information.
- **Panda et al. (2024)**, “Lottery Ticket Adaptation: Mitigating Destructive Interference in LLMs,” ICML 2024, arXiv:2406.16797. LoTA enables sparse multi-task fine-tuning. *Connection*: our selective channel tuning (Phase 5) is architecturally motivated LoTA—the “ticket” is the social channel, identified by role rather than by pruning.

9.3 Renormalisation Group and Percolation in Deep Learning

These papers provide the statistical mechanics foundations that connect to our criticality framing.

- **Mehta & Schwab (2014)**, “An exact mapping between the Variational Renormalization Group and Deep Learning,” arXiv:1410.3831. Foundational paper establishing that RBMs perform coarse-graining equivalent to variational RG. *Connection*: layers of our model perform successive coarse-graining of the input; the social field adds a lateral channel that carries state *across* the RG flow rather than just *through* it.
- **Li & Wang (2018)**, “Neural Network Renormalization Group,” arXiv:1802.02840. Variational RG using normalising flows. *Connection*: the field’s evaporation/deposit dynamics define a flow in state space; the critical ε is where this flow neither diverges nor collapses—the fixed point.
- **Redman et al. (2022)** (cited above under LTH). RG perspective on pruning universality. Most directly relevant RG-in-neural-networks paper for our framing.
- **arXiv:2512.13853 (2025)**, “Dropout Neural Network Training Viewed from a Percolation Perspective.” Dropout as percolation on the network graph. *Connection*: the social field’s connectivity (which tokens’ deposits reach which future queries) defines a directed percolation process. Evaporation rate controls the percolation threshold. At critical ε , information percolates across the full sequence; above it, clusters fragment.
- **arXiv:2602.15224 (2026)**, “Phase Transitions in Neural Network Pruning.” Pruning as percolation: functional phase persists until $\sim 1\%$ remaining edges, then sharp collapse. *Connection*: analogous to the sharp ε threshold we observe—the social field provides a functional channel until evaporation exceeds a critical rate, then it collapses.
- **Wei et al. (2022)**, “Emergent Abilities of Large Language Models,” arXiv:2206.07682. Emergent abilities as discontinuous jumps in performance with scale. *Connection*: our social field may lower the scale threshold for emergence of state-tracking abilities. A 30M model with a field might exhibit tracking behaviour that requires 100M+ without it.
- **arXiv:2102.06701 (2021)**, “Explaining neural scaling laws.” Power-law scaling from variance-limited and resolution-limited regimes. *Connection*: the social field may change the scaling exponent for state-dependent tasks by removing an architectural bottleneck (lack of persistent state), shifting the model from resolution-limited to variance-limited regime at smaller scale.

Synthesis note. The unifying thread: *criticality as an organising principle at multiple levels*. Grokking is a phase transition in learning dynamics. Lottery tickets emerge at a pruning phase transition. The social field operates at a critical evaporation rate. RG provides the mathematical framework linking all three—each is a system poised at a fixed point where information is neither created nor destroyed, but transformed across scales. The social field makes this explicit in the architecture: evaporation is the RG scale parameter, deposits are the relevant operators, and the critical ε is the fixed point where long-range correlations (state tracking) emerge.



Figure 4: Per-token loss on four entity-tracking stories, with critical tokens highlighted (tokens requiring memory of earlier entities or events). Top subplot of each pair: per-position cross-entropy for baseline (blue) and social (orange). Bottom subplot: difference (positive = social better). Results are mixed: the wizard/stone story shows strong social advantage at critical tokens (+0.429), while ball/key stories show baseline advantage at critical tokens.

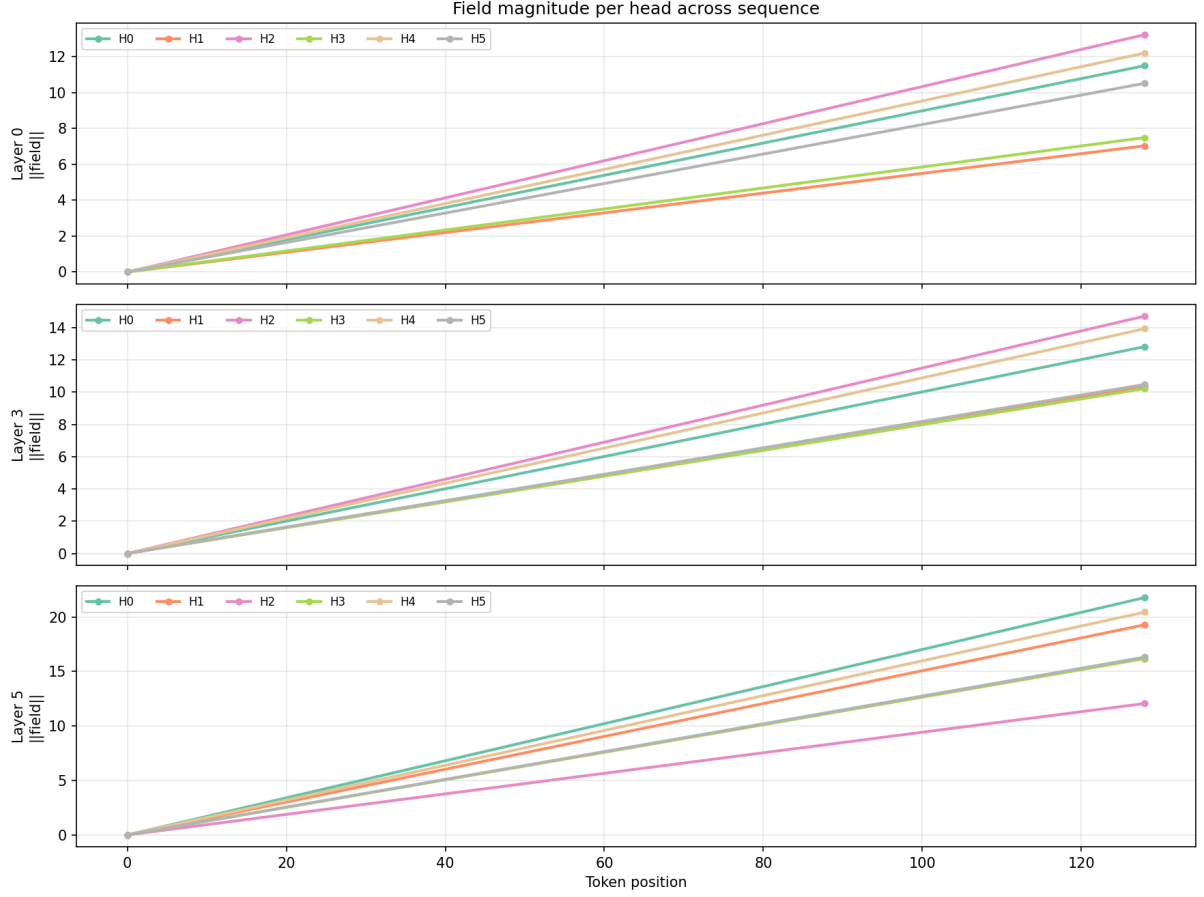


Figure 5: Social field magnitude ($\|F\|_2$) per attention head across sequence positions, shown for layers 0, 3, and 5. Each line is one head. Field magnitude grows approximately linearly within chunks, with different heads accumulating at different rates—evidence of emergent functional specialisation.

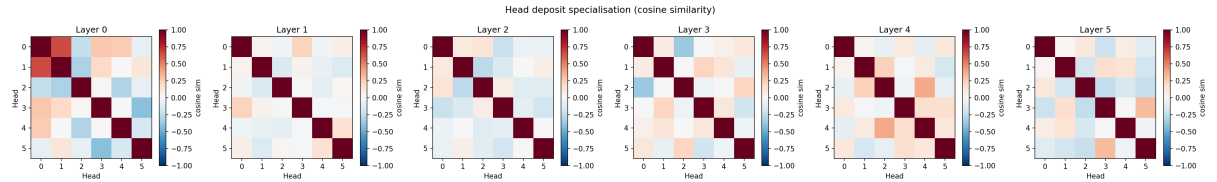


Figure 6: Cosine similarity of average deposit directions between heads, shown per layer. By layer 3+, off-diagonal similarity drops near zero, indicating that heads deposit into orthogonal subspaces of the field. This is emergent typed communication: different heads carry different “pheromone types” without explicit allocation.

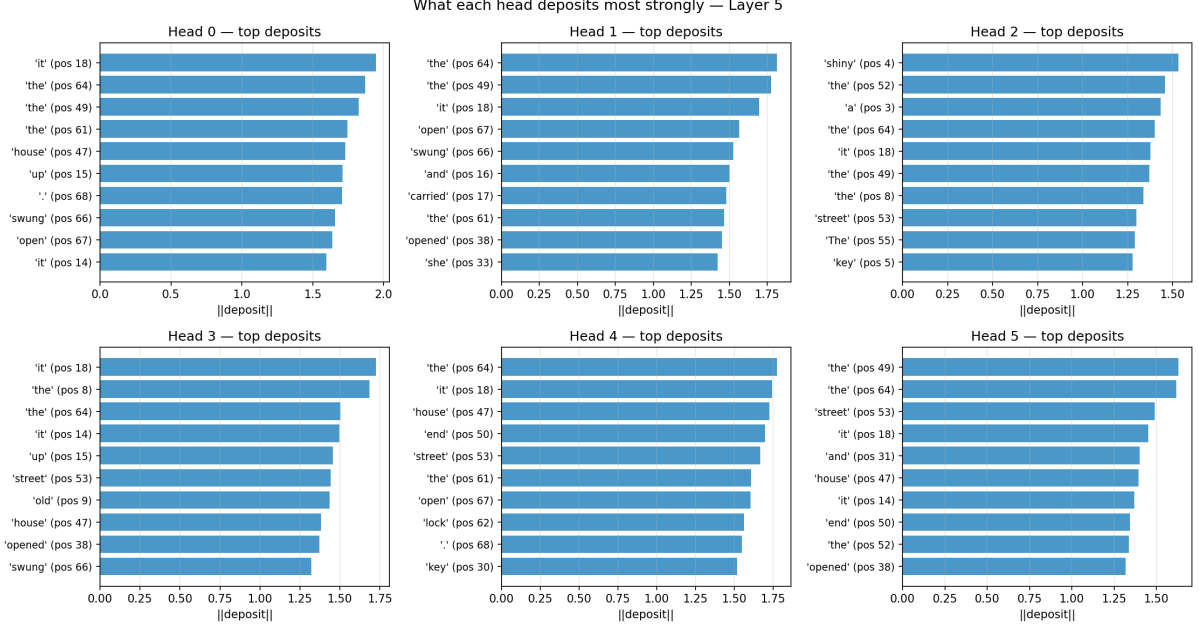


Figure 7: Top-10 tokens by deposit magnitude for each head in layer 5. Different heads respond most strongly to different token types: content words (nouns, verbs), function words (articles, prepositions), or punctuation. This specialisation is fully emergent.

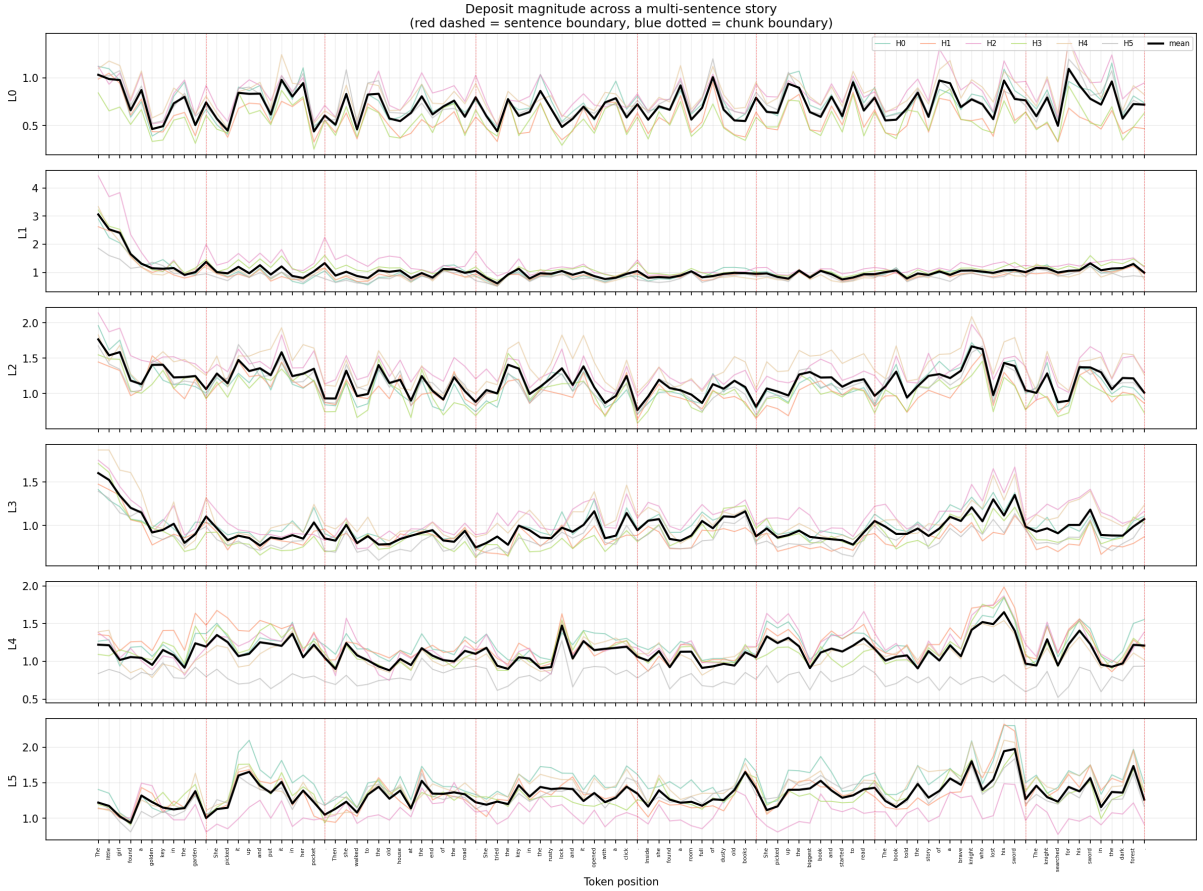


Figure 8: Deposit magnitude across token positions with sentence boundaries marked. Deposits spike at semantically important tokens (character names, key actions) and at structural boundaries (sentence starts, clause transitions). The field thus tracks both content and structure.

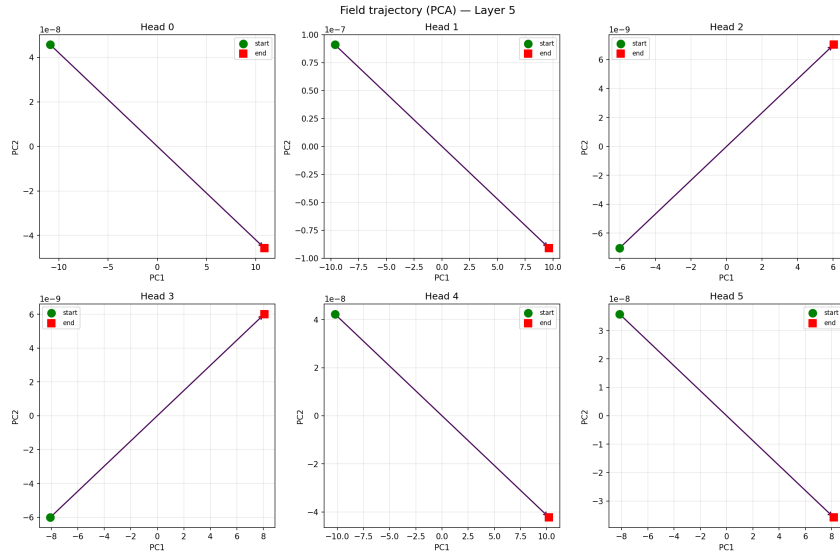


Figure 9: PCA projection of the field state trajectory in layer 5. Each point is the field state after processing one chunk. The trajectory traces a smooth path through the 2D projection, with narrative events producing distinct directional shifts. Different heads (colours) traverse different regions of the space.